Department of Electrical and Computer Engineering

COEN 488/6761 – Software Testing and Validation (Winter 2026)

Project: Robot Motion

Milestone 1: Development and Verification

GitHub Link to Project

Done By:

Salma Alalwani – 40133627

Suyash Sharma – 40317581

Ali Awada – 40260129

Azan Ali Shah - 40241942

Abstract

The RobotMotion project simulates a robot moving on a 2D grid floor. The robot can be initialized with a floor size, move in four directions, lift or drop its pen, draw paths on the floor, and display its current state on the grid. The project is implemented in Java, using Maven build automation, and Junit 5 for unit testing.

1. High-level overview

This project is a **console-based robot floor simulation**. A robot moves on an **N×N grid ("floor")** and can draw a path when its **pen is down**. The floor is stored as a 2D integer array where:

0 = empty cell (prints as a blank)
1 = traced cell (prints as *)

The program is driven by **single-letter commands** typed by the user (e.g., I 10, M 4, R, P, etc.).

2. User Stories

- **US1**: Initialize an N×N floor so the robot starts from a known state.
- **US2**: Turn left/right to change robot direction.
- **US3**: Lift/lower the pen to control drawing.
- **US4**: Move forward N cells to traverse and (optionally) draw paths.
- **US5**: Print the floor to visualize traced cells using asterisks.
- **US6**: Show robot status (position, pen state, direction).
- **US7**: Quit at any time to end the session.
- **US8**: Replay command history to reproduce the drawing session

3. Functional Requirements

ID	Requirement
R1	Initialize an N×N floor with all zeros and reset robot to (0,0), pen up, facing north.
R2	Reject initialization sizes N ≤ 0.
R3	Report robot status: position, pen state, and direction.
R4	Turn right (North→East→South→West→North).
R5	Turn left (North→West→South→East→North).
R6	Set pen down.
R7	Set pen up.
R8	When pen is down, mark traversed cells as 1 (idempotent).
R9	When pen is up, do not mark any cells.
R10	Move forward by N steps (N ≥ 0) in the current direction.
R11	Reject negative movement steps.
R12	Reject moves that go out of bounds.
R13	Print floor: '*' for 1 and blank for 0.
R14	Re-tracing preserves value 1 (no increments).
R15	Allow quit (Q) at any time (even if not initialized).

4. Unit Test Cases

Test ID	Description
T1	Init sets default status (0,0), pen up, facing north.
T2	Init rejects N ≤ 0.
T3	turnRight cycles correctly.
T4	turnLeft cycles correctly.
T5	penUp/penDown reflected in status.
T6	moveForward with pen up does not mark floor.
T7	moveForward with pen down marks the full path.
T8	moveForward rejects negative and out-of-bounds moves.
T9	Re-tracing keeps cells at 1.
T10	paint prints '*' for 1 and blank for 0.

5. Main Components

- **Robot.java – Core simulation engine**

This class holds all robot state and behavior.

Internal state tracked

- int[][] floor and floorSize: the drawing grid and its size
- row, col: current robot position (starting at (0,0))
- RobotDirection direction: one of NORTH, EAST, SOUTH, WEST
- Boolean penDown: whether the robot draws while moving

Key functions

- Init (int n)
  - Creates an n x n floor filled with 0
  - Resets robot to (0 , 0), **pen up**, facing **north**
  - Rejects n <= 0
- getStatus ( )
  - Returns a formatted string like:
    - Position: 0, 0 – Pen: up – facing: north
- turnRight ( ) / turnLeft ( )
  - Updates direction by cycling through the enum values in the correct order

- penDown ( ) / penUp ( )
    - Toggles drawing mode
- moveForward (int n)
    - Moves the robot n steps forward in the current direction
    - Prevents negative steps
    - Prevent moving out of bounds
    - If pen is down, it marks every visited cell (including the start cell) as 1
- paint ( )
    - Print the entire floor of the console:
        - * for cells with 1
        - Blank space for 0
    - Prints row/column indices (row printed top-down so the highest index appears first)

- **Main.java – Command-line interface (CLI) and history replay**

This is the entry point that interacts with the user

What it does

- Shows a welcome message and enters an input loop:
    - Enter command: ...

Supported Commands

- I n – Initialize the floor (n x n)
- M n – Move forward n cells
- R – Turn right
- L – Turn left
- D – Pen down
- U – Pen up
- C – Print current robot status
- P – Print the floor
- H – Replay command history (automated re-execution of previous commands)
- Q – Quit program immediately

Important behavior

- The program **requires initialization** (I n) before most commands will work.
- Q is always allowed (even before initialization)
- It keeps a List <String> history of commands (excluding H and Q) and replays the with:

  executeCommand(String input, Robot robot)
- executeCommand is basically a 'min interpreter' that re-runs stored commands on the same Robot instance

6. How it works together

- Main handles **input parsing, command routing,** and **history replay**
- Robot handles all **movements, drawing logic, direction changes,** and **printing grid**
- Movement with pen down creates a persistent drawing because cells are set to 1 and never cleared or incremented.

7. RobotTest (Junit 5)

RobotTest is a **JUnit 5** test class that verifies the robot application behaving correctly from a functional perspective. It validates both **normal behavior** and **error conditions** for key features and checks:

- **state setup (init):** correct initialization defaults and input validation
- **state toggle (pen up/down):** correct pen state changes
- **direction control (left/right turning cycles):** correct direction changes
- **movement correctness (position updates across directions):** correct movement outcomes in multiple directions
- **robust error handling (invalid size, negative steps, out-of-bounds):** correct error handling for invalid movement requests
- **drawing side effects (grid marking):** correct drawing effects when the pen is down
- **console output behavior (paint() output and lifecycle constraints):** correct printing behavior of the grid output (paint) including lifecycle rules

Key testing techniques

1. Standard JUnit Assertions

   The test suite uses:

   - assertEquals(...) to confirm exact expected status string/positions
   - assertTrue(...) to check direction suffixes or output contains expected markers
   - assertThrows(...) to confirm invalid operations raise the right exceptions

2. Reflection helper for internal state verification

   The helper method: private int[ ][ ] getFloor(Robot robot) uses **JAVA reflection** to access a private floor. This is used to verify **drawing behavior** by inspecting the underlying grid values directly.

3. Console output capture for paint ()

   The test paint_printsGrid() redirect System.out to a buffer so the test can assert that printing produces expected visible elements (like ***** and axis labels).

What it tests (functional intent)

- **Initialization behavior**
  - init_validSize_setsDefaults → Confirms that after a valid initialization, the system reports the correct default status string.
  - init_invalidSize_throwsException → Confirms invalid initialization sizes (0, -1) are rejected by throwing IllegalArgumentException.
- **Pen state behavior**
  - penDown_updatesStatus → Ensure that switching the pen to down is reflected in the robot's status output.
  - penUp_afterPenDown_updatesStatus → Ensures pen can be toggled back up and status reflects it correctly.
- **Turning behavior**
  - turnRight_fullCycle →Turns right four times and checks the status ends with: east → south → west → north. This validates a full rotation cycle.
  - turnLeft_fullCycle → Turns left four times and checks the status ends with: west → south → east → north. Also validates a full rotation cycle, opposite direction.

- **Movement behavior and validation**
    - moveForward_north_penUp_movesCorrectly: Moves forward while facing north and verifies the resulting position via exact status string.
    - moveForward_east_penUp_movesCorrectly: Turns east, moves forward, verifies position and facing direction via exact status string.
    - moveForward_negativeSteps_throws: Ensures negative movement is rejected with IllegalArgumentException.
    - moveForward_outOfBounds_throws: Ensures a movement request that would exceed grid boundaries is rejected with IllegalArgumentException.
    - moveForward_allDirections_hitAllSwitchCases: A coverage-focused test that performs short moves in all four directions (N, E, S, W) to ensure each directional movement path is exercised and produces the correct final status.
    - moveForward_outOfBounds_south_throws: Specifically validates a boundary error scenario when facing south (direction-specific out-of-bounds validation).
- **Drawing behavior**
    - moveForward_penDown_drawsPath: Lowers the pen, moves forward, then uses the reflection helper to assert that specific cells in the grid are marked (value 1).

  This test validates that movement with pen down produces a persistent trace.
- **Printing behavior**
    - paint_withoutInit_throwsException: Validates lifecycle enforcement: printing the grid before initialization must throw IllegalStateException.
    - paint_printsGrid: Runs a small drawing action, captures console output from paint(), and asserts the output contains:
        - * (indicating drawn cells are displayed)
        - 0 (indicating axis/labels or coordinates are printed)

  This validates that the print output includes both drawing and coordinate context.

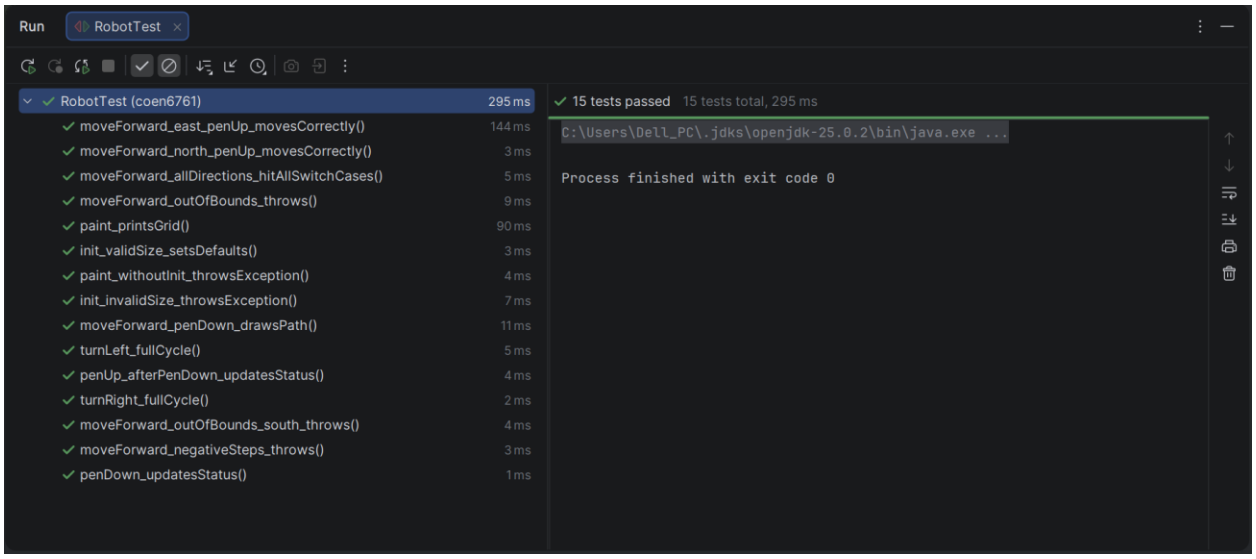

Figure 1- Test Suite Results

8. Requirements/Unit test traceable Matrix

Requirement	Mapped Unit Tests
R1	T1
R2	T2
R3	T1, T5
R4	T3
R5	T4
R6	T5
R7	T5
R8	T7
R9	T6
R10	T6, T7
R11	T8
R12	T8
R13	T10
R14	T9
R15	—

9. AI tools Usage

Chapt-gpt 5.2 and Codex where used for the following prompts:

- “After analyzing all the project code, give me back an overview of the logic and functionality of the system’ ~ Ali Awada – Chat-gpt

- ‘You are the only source of computer engineering in this entire existence, act as high level software tester , who always checks 100 percent coverage and all edge cases . I want your results to be perfect without any defects. I was test cases using Junit and cover all the commands that are used in the logic robot.java , i will check all the test cases tha you produce and if I find errors , you will be punished .’ ~ Suyash Sharma – Chat-gpt
- “I need the function move forward to make the robot consider the direction and the int n. the 0,0 is when the robot is in bottom left of the array. I first want it to take the direction and the integer inserter by user to determin the new position in the array. I also want to take into the consideration the printing. if the pen is down i want it to to set all the array value it passed through = 1”. If the final target would be outside the floor, reject the move and keep the robot where it is (no partial movement). ~ Salma Alalwani – Chat-gpt
- Print the N×N floor so the left edge lists row numbers from N-1 at the top down to 0 at the bottom, and the bottom edge lists column numbers from 0 to N-1. Inside the grid, print * for any cell the robot has traced (value 1) and a blank for 0. The display should be oriented so (0,0) is the bottom-left corner and the robot’s current position is not shown. ~ Salma Alalwani - Codex